INFORME DE GUÍA PRÁCTICA #1

I. PORTADA

Tema:	Pilas y colas (implementación)
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Tercero B
Alumnos participantes:	Nata Analuiza Walter Alexis
Asignatura:	Estructura de Datos
Docente:	Ing. Jose Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

- Analizar situaciones problemáticas reales y computacionales en las que el orden de inserción y extracción de elementos determina la solución, con el fin de identificar la utilidad de las estructuras de datos tipo pila y cola antes de su implementación en lenguaje C++.

Específicos:

- Investigar los fundamentos conceptuales de las estructuras de datos tipo pila y cola.
- Diferenciar el comportamiento LIFO y FIFO mediante el análisis de casos prácticos.
- Representar secuencialmente la entrada y salida de elementos en problemas concretos.
- Formular soluciones preliminares utilizando diagramas, tablas de seguimiento y pseudocódigo.
- Desarrollar criterio analítico para seleccionar la estructura adecuada según el problema planteado.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 4
No presenciales: 0

2.4 Instrucciones

1. El trabajo se desarrollará de manera individual.
2. La redacción debe ser formal, clara y coherente, utilizando lenguaje técnico apropiado. – No se aceptarán definiciones copiadas textualmente sin explicación propia del estudiante.
3. Se valorará el análisis, la argumentación y la correcta representación de los casos de estudio.
4. El producto final deberá entregarse en formato Word o PDF, según la indicación del docente.

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial, TAC
- Computador, Cuadernos, esferos, lápiz, borrador, Aula virtual e Internet

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

☒Plataformas educativas

☒Simuladores y laboratorios virtuales
☒Aplicaciones educativas
☐Recursos audiovisuales
☐Gamificación
☒Inteligencia Artificial
Otros (Especifique): _____

2.5 Actividades por desarrollar

Investigación Conceptual

Realice una revisión bibliográfica y explique con sus propias palabras los siguientes conceptos:

Estructura de datos lineal: Las estructuras de datos lineales guardan la información de forma ordenada, una detrás de otra, lo que hace más fácil recorrer y trabajar con los datos. Con ellas podemos acceder, agregar o eliminar elementos en posiciones específicas sin complicarte mucho. Además, suelen usar memoria continua, lo que ayuda a que el acceso sea rápido, aunque a veces no se aprovecha todo el espacio.[1]

Ejemplo: Una lista de números [10, 20, 30, 40], donde podemos recorrerlos en orden desde el primero hasta el último.

Pila: Una pila es una estructura de datos lineal donde los elementos se organizan uno sobre otro. Las operaciones de inserción y eliminación se realizan siempre en el mismo extremo, llamado tope. Funciona bajo el principio LIFO (último en entrar, primero en salir). Esto significa que el elemento más reciente es el primero en salir. Se usa para invertir el orden de los datos y aparece en ejemplos como pilas de libros o el historial del navegador.[2]

Ejemplo: Una pila de libros, si ponemos 3 libros, el último que pusimos es el primero en sacar.

Cola: Una cola es una estructura de datos lineal donde los elementos se organizan en secuencia. Funciona bajo el principio FIFO (primero en entrar, primero en salir). Esto significa que el primer elemento agregado es el primero en eliminarse. Las inserciones se hacen al final y las eliminaciones al inicio de la cola. Se usa en situaciones como filas de personas o gestión de tareas en sistemas.[3]

Ejemplo: Una fila en un banco, la primera persona en llegar, es la primera en ser atendida.

Principio LIFO: LIFO (Last In, First Out) es un método donde el último elemento en entrar es el primero en salir. Se usa en inventarios para registrar primero los productos más recientes como vendidos. Esto hace que los costos más nuevos se reflejen antes en los gastos. Los productos más antiguos quedan como inventario disponible. Es lo contrario de FIFO, donde los primeros en entrar son los primeros en salir.[4]

Ejemplo: Si apilamos los platos, el último que colocaste arriba es el primero que vamos a retirar.

Principio FIFO: FIFO (First In, First Out) es un método donde el primer elemento en entrar es el primero en salir. Se usa para evitar que productos se dañen o queden obsoletos con el tiempo. Los artículos más antiguos se venden o utilizan antes que los nuevos. Es común en productos como alimentos, medicamentos o ropa. Ayuda a mantener una buena rotación y evitar pérdidas en inventarios.[5]

Ejemplo: En una fila para comprar comida, el primero en llegar es el primero en recibir su pedido.

Operación de inserción: La inserción es el proceso de agregar un nuevo elemento a una estructura de datos. Implica ubicarlo en la posición correcta según la organización de la estructura. Es una operación básica en la gestión de datos. Influye en qué tan rápido se pueden usar y modificar los datos. Es clave para mantener el orden y el funcionamiento de la estructura.[6]

Ejemplo: Agregar el número 25 en una lista [10, 20, 30] para que quede [10, 20, 25, 30].

Operación de eliminación:
La eliminación es el proceso de quitar un elemento de una estructura de datos. Puede implicar reorganizar la estructura para mantener su orden y funcionamiento. Permite liberar el espacio que ocupaba el dato eliminado. También requiere ajustar enlaces, punteros o posiciones según la estructura. Es clave para mantener la integridad y eficiencia de los datos.

Ejemplo: Quitar el número 20 de la lista [10, 20, 30], quedando [10, 30].

Implementación mediante arreglos: Un arreglo es una estructura de datos que almacena elementos del mismo tipo en memoria continua. Permite acceder a cada elemento mediante un índice numérico. Facilita el manejo de grandes cantidades de datos de forma ordenada. Puede ser unidimensional, bidimensional o dinámico según su uso. Es muy usado por su rapidez de acceso y simplicidad en programación.[7]

Ejemplo: Un arreglo int numeros[3] = {1, 2, 3}, donde cada número se accede por su posición.

Implementación mediante nodos: Un nodo es un elemento de una estructura de datos que puede guardar distintos tipos de información, además del dato, contiene uno o más enlaces hacia otros nodos. Estos enlaces, llamados punteros, sirven para conectar los nodos entre sí. Si un enlace es null, significa que no apunta a ningún otro nodo. Los nodos son la base de estructuras como listas enlazadas, árboles y grafos.[8]

Ejemplo: Una lista enlazada donde cada nodo tiene un valor y apunta al siguiente: 5 → 10 → 15.

ANÁLISIS DEL CASO 1: PARÉNTESIS BALANCEADOS

Analice el problema de validación de expresiones con símbolos de apertura y cierre. Trabaje con las siguientes expresiones:

(a+b) * (c-d):

1. Encuentro (, lo inserto en la pila = Pila: [(]
2. Leo a+b , no afecta la pila.
3. Encuentro) , coincide con el tope (, elimino = Pila: []
4. Encuentro (, inserto , Pila: [(]
5. Leo c-d , no afecta.
6. Encuentro), coincide con (, elimino, Pila: []

La pila queda vacía, esta corresponde a una Estructura Pila (LIFO).

[(a+b) * {c/d}]:

1. Encuentro [, inserto, Pila: [[]
2. Encuentro (, inserto, Pila: [[, (]

3. Leo a+b, no afecta.
4. Encuentro), coincide con (, elimino, Pila: [[]
5. Encuentro {, inserto, Pila: [[, {]
6. Leo c/d → no afecta.
7. Encuentro }, coincide con {, elimino, Pila: [[]
8. Encuentro], coincide con [, elimino, Pila: []

La pila queda vacía, Expresión válida, la estructura Pila (LIFO).

(a+b]:

1. Encuentro (, inserto, Pila: [(]
2. Leo a+b, no afecta.
3. Encuentro], el tope es (, pero no coincide, tengo un error.

Expresión inválida la apertura y cierre no corresponden, la estructura es Pila (LIFO), pero aquí hay error.

((a+b):

1. Encuentro (, inserto, Pila: [(]
2. Encuentro (, inserto, Pila: [(, (]
3. Leo a+b, no afecta.
4. Encuentro), coincide con (, elimino = Pila: [(]
5. Fin de la expresión, queda un (sin cerrar.

Expresión inválida porque faltan cierres, la estructura adecuada es Pila (LIFO), que permite detectar el error.

ANÁLISIS DEL CASO 2:

Atención en ventanilla Estructura de Datos

Considere un sistema de atención al cliente en el que llegan las siguientes personas en este orden: Ana, Luis, Pedro, Marta y José.

Paso	Acción	Estado de la cola
0	Llegan Ana, Luis, Pedro, Marta, José	[Ana, Luis, Pedro, Marta, José]
1	Atienden 3 clientes (Ana, Luis, Pedro)	[Marta, José]
2	Llega Carla	[Marta, José, Carla]
3	Atienden 2 clientes (Marta, José)	[Carla]

El sistema de atención funciona con el principio FIFO, o sea, el primero que llega es el primero en ser atendido.
La estructura que mejor representa esto es una cola, porque es igual a una fila en la vida real.
Si en lugar de eso se usara una pila, pasaría algo raro: Carla sería atendida antes que Marta y José, aunque llegó después, lo cual no tiene sentido en este caso.

COMPARACIÓN ANALÍTICA DE LOS CASOS

1. Diferencias en el procesamiento de los elementos
En el caso de los paréntesis, cada símbolo abierto debe cerrarse antes que los anteriores, así que se necesita controlar bien el orden. Por eso se usa una pila (LIFO).
En el caso de la atención en ventanilla, los clientes se atienden según el orden de llegada: el primero en entrar es el primero en salir, lo que se refleja con una cola (FIFO).

2. ¿Dónde el último elemento ingresado sale primero?
En los paréntesis balanceados, porque el último símbolo abierto es el que debe cerrarse primero.

3. ¿Dónde el primer elemento ingresado sale primero?
En la atención en ventanilla, ya que el primer cliente que llega es el primero en ser atendido.

4. Errores si se usa la estructura equivocada
Si se usara una cola para los paréntesis, los cierres no coincidirían y se podrían aceptar expresiones inválidas.
Si se usara una pila en la atención al cliente, los últimos en llegar serían atendidos antes que los que llegaron primero, rompiendo la lógica de la fila.

5. ¿Por qué es importante analizar el problema antes de programar?
Porque cada situación tiene su propia forma de procesar los elementos.
Sin un análisis previo, podemos elegir la estructura equivocada y cometer errores. Analizar primero asegura que tu solución en C++ funcione correctamente y de manera eficiente, respetando la lógica real del problema.

PROPUESTA DE SOLUCIÓN LÓGICA

```
ALGORITMO VERIFICARPARENTESISBALANCEADOS

   Crear Pila vacía

   Para cada símbolo en expresion hacer
      Si símbolo es de apertura ( (, [, { ) entonces
         Insertar símbolo en la Pila
      Si símbolo es de cierre ( ), ], } ) entonces
         Si Pila está vacía entonces
            Mostrar "Expresión inválida"
            Terminar algoritmo
         FinSi

         Consultar tope de la Pila
         Si tope coincide con el tipo de cierre entonces
            Eliminar tope de la Pila
         Sino
            Mostrar "Expresión inválida"
            Terminar algoritmo
         FinSi
      FinSi
   FinPara

   Si Pila está vacía entonces
      Mostrar "Expresión válida"
   Sino
      Mostrar "Expresión inválida"
   FinSi
FinAlgoritmo
```

SIMULACIÓN DE ATENCIÓN EN VENTANILLA

```
Algoritmo AtencionVentanilla

   Crear Cola vacía

   // Llegada inicial de clientes
   Insertar(Ana)
```

```
    Insertar(Luis)
    Insertar(Pedro)
    Insertar(Marta)
    Insertar(José)

    // Se atienden los tres primeros
    Para i desde 1 hasta 3 hacer
       Si Cola no está vacía entonces
          cliente ← Consultar primer elemento de la Cola
          Eliminar primer elemento de la Cola
          Mostrar "Atendido: " + cliente
       FinSi
    FinPara

    // Llega Carla
    Insertar(Carla)

    // Se atienden dos más
    Para i desde 1 hasta 2 hacer
       Si Cola no está vacía entonces
          cliente ← Consultar primer elemento de la Cola
          Eliminar primer elemento de la Cola
          Mostrar "Atendido: " + cliente
       FinSi
    FinPara

    // Estado final
    Mostrar "Clientes restantes en la cola: "
    Mientras Cola no esté vacía hacer
       Mostrar Consultar primer elemento de la Cola
       Eliminar primer elemento de la Cola
    FinMientras
FinAlgoritmo
```

2.6 Habilidades blandas empleadas en la práctica

☒ Trabajo en equipo
☐ Responsabilidad

2.7 Conclusiones

- El análisis de estos problemas muestra que la elección de la estructura de datos depende totalmente de cómo entran y salen los elementos. No hay una solución única; cada caso necesita un modelo específico.
- Para los paréntesis balanceados, la estructura correcta es la pila LIFO, porque el último símbolo que se abre es el primero que se debe cerrar. Esto asegura que las expresiones se validen correctamente.
- Para la atención en ventanilla, lo adecuado es usar una cola FIFO, ya que el primer cliente en llegar debe ser el primero en ser atendido, manteniendo un orden justo y lógico.
- Si se usa la estructura equivocada, surgen errores: por ejemplo, una cola para los paréntesis permitiría cierres que no corresponden, y una pila en la ventanilla cambiaría el orden de atención de los clientes, generando caos.

2.8 Recomendaciones

- Elegir la estructura de datos correcta depende de entender cómo entran y salen los elementos. Esto ayuda a evitar errores y asegura que la solución sea eficiente.
- Trabajar con casos como los paréntesis balanceados o la atención en ventanilla permite ver claramente cómo funcionan las pilas y las colas tanto en la vida real como en la programación.
- Mostrar el estado de la pila o la cola en cada paso hace que sea más fácil de entender, aprender y también mejora la presentación del trabajo académico.
- Es importante practicar acciones como insertar, eliminar, verificar si la estructura está vacía y consultar el siguiente elemento antes de empezar a programar en C++.

2.9 Referencias bibliográficas

[1] "¿Qué es la estructura de datos lineal? Todo lo que debes saber." Accessed: Mar. 27, 2026. [Online]. Available: https://www.theknowledgeacademy.com/blog/what-is-linear-data-structure/

[2] "3.3. ¿Qué es una pila? — Solución de problemas con algoritmos y estructuras de datos." Accessed: Mar. 27, 2026. [Online]. Available: https://runestone.academy/ns/books/published/pythoned/BasicDS/QueEsUnaPila.html

[3] "Estructura de datos de cola: tipos, operaciones, ejemplos." Accessed: Mar. 27, 2026. [Online]. Available: https://www.wscubetech.com/resources/dsa/queue-data-structure

[4] "Comprensión de LIFO: método de inventario de último en entrar, primero en salir." Accessed: Mar. 27, 2026. [Online]. Available: https://www.investopedia.com/terms/l/lifo.asp

[5] "¿Qué es el método FIFO? - Blog IEP." Accessed: Mar. 27, 2026. [Online]. Available: https://iep.edu.es/que-es-el-metodo-fifo/

[6] "Inserción: Guía de estudio de estructuras de datos | Fiveable." Accessed: Mar. 27, 2026. [Online]. Available: https://fiveable.me/data-structures/key-terms/insertion

[7] "¿Qué son los arreglos en programación? - Onmex." Accessed: Mar. 27, 2026. [Online]. Available: https://onmex.mx/tecnologia-y-desarrollo/que-son-los-arreglos-en-programacion/

[8] "Comprender los nodos en las estructuras de datos | Codecademy." Accessed: Mar. 27, 2026. [Online]. Available: https://www.codecademy.com/article/nodes-data-structures-and-algorithms

2.10 Anexos